De Casteljau Algorithm: Quartic (3-D)

CairoMakie Visualisation

Code and Plot Explanation for Julia Beginners

`22-de-casteljau-makie.jl`

E.R. Nurwijayadi

2026-03-15

Directed and curated by E.R. Nurwijayadi. Code and explanations AI-generated.

This document explains `22-de-casteljau-makie.jl`. This is the most complex script in the series so far: the De Casteljau algorithm applied to a *quartic* (degree-4) curve in 3-D space. A quartic requires **four** reduction levels instead of two. New concepts: the general `reduce_level` function, `getfield` for dynamic named-tuple access, colour-with-alpha tuples, and NaN dummy lines for legend entries.

Contents

1 The Plot **2**

2 The De Casteljau Algorithm on a Quartic Curve **2**

3 Reading the Plot **3**
3.1 The layers . . . 3
3.2 Worked example: $u = 0.50$ (orange) . . . 3

4 Data and Constants **4**

5 The Math Functions **4**
5.1 `lerp3` — renamed linear interpolation . . . 4
5.2 `reduce_level` — the general reduction step . . . 5
5.3 `dc_steps` — four reductions . . . 5
5.4 `dc_point` and `dc_curve` . . . 6

6 `draw_and_save` — New Drawing Techniques **6**
6.1 NaN dummy lines for legend entries 6
6.2 The snapshot loop: `getfield` and colour-with-alpha 7
6.3 The final point T0 8

7 Type Map **9**

8 Entry Point **9**

1. The Plot

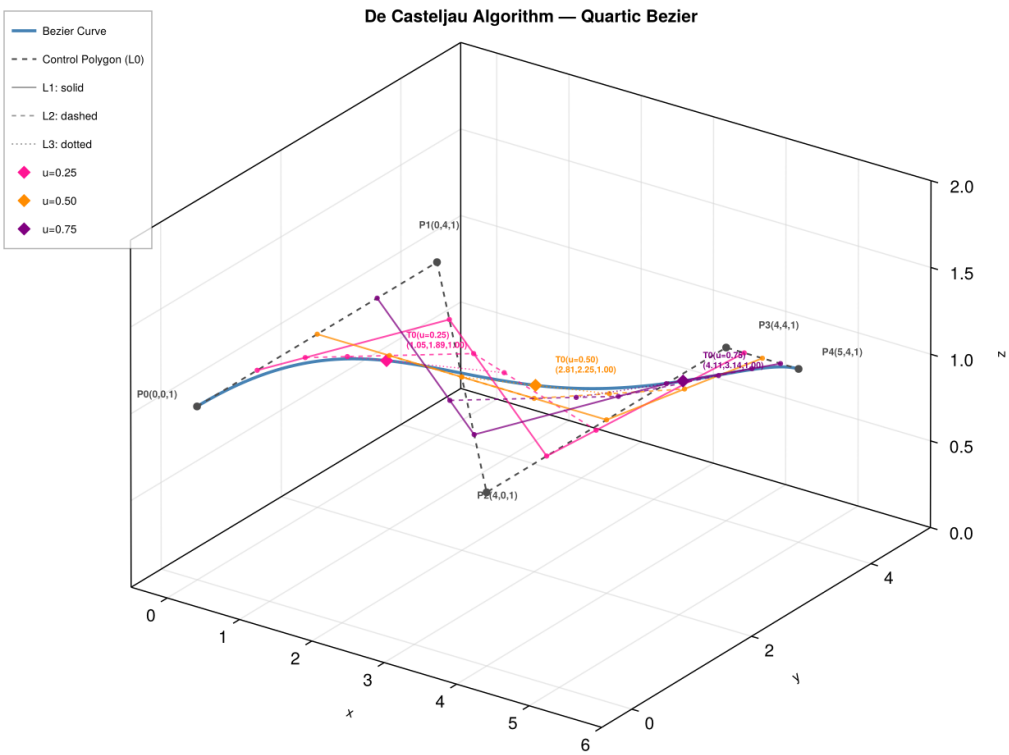


Figure 1: De Casteljau algorithm on a quartic Bézier in 3-D. Three snapshots at $u = 0.25$ (pink), $u = 0.50$ (orange), $u = 0.75$ (purple). Each snapshot shows levels L1 (solid), L2 (dashed), L3 (dotted), and the final curve point T0 (diamond).

2. The De Casteljau Algorithm on a Quartic Curve

Script 12 showed De Casteljau on a quadratic curve — two control points became one in two steps. A quartic curve has five control points, so the algorithm needs four reduction steps.

For a quartic Bézier with control points P_0, P_1, P_2, P_3, P_4, the De Casteljau algorithm at parameter u proceeds as follows:

Level 0 (L0): 5 points — the original control polygon.

Level 1 (L1): 4 points — interpolate adjacent pairs from L0:

$$Q_i^1 = (1-u)\,P_i + u\,P_{i+1} \quad i = 0, 1, 2, 3$$

Level 2 (L2): 3 points — interpolate adjacent pairs from L1:

$$Q_i^2 = (1-u)\,Q_i^1 + u\,Q_{i+1}^1 \quad i = 0, 1, 2$$

Level 3 (L3): 2 points — interpolate adjacent pairs from L2:

$$Q_i^3 = (1-u)\,Q_i^2 + u\,Q_{i+1}^2 \quad i = 0, 1$$

Level 4 (L4): 1 point — the final curve point:

$$T_0 = (1-u)\,Q_0^3 + u\,Q_1^3$$

Each level is one row shorter than the previous. The pattern "n points $\to n-1$ points" is exactly what `reduce_level` implements.

> **Generalisation.** This same pattern works for any degree n: apply `reduce_level` exactly n times. The code is identical for quadratic, cubic, quartic, or any higher degree — only the number of calls changes.

3. Reading the Plot

3.1. The layers

Blue curve — the quartic Bézier, 300 sample points.

Gray dashed polygon — L0, the original 5 control points connected by dashed lines. Labelled P0–P4.

Coloured solid lines (L1) — for each snapshot colour, the 4 L1 interpolation segments.

Coloured dashed lines (L2) — the 3 L2 segments.

Coloured dotted lines (L3) — the 2 L3 segments.

Coloured diamonds (T0) — the final L4 point — the curve point at that u value, sitting exactly on the blue curve.

The legend has two kinds of entries: line style entries (L1 solid, L2 dashed, L3 dotted) shown in grey, and point entries (the three diamonds) shown in their snapshot colours.

3.2. Worked example: $u = 0.50$ (orange)

At $u = 0.5$ all interpolations are midpoints.

> **L1** (midpoints of L0 edges):
>
> $$\begin{aligned} Q_0^1 &= \tfrac{1}{2}(0,0,1) + \tfrac{1}{2}(0,4,1) = (0,2,1) \\ Q_1^1 &= \tfrac{1}{2}(0,4,1) + \tfrac{1}{2}(4,0,1) = (2,2,1) \\ Q_2^1 &= \tfrac{1}{2}(4,0,1) + \tfrac{1}{2}(4,4,1) = (4,2,1) \\ Q_3^1 &= \tfrac{1}{2}(4,4,1) + \tfrac{1}{2}(5,4,1) = (4.5,4,1) \end{aligned}$$

L2 (midpoints of L1 edges):

$$\begin{aligned} Q_0^2 &= \tfrac{1}{2}(0,2,1) + \tfrac{1}{2}(2,2,1) = (1,2,1) \\ Q_1^2 &= \tfrac{1}{2}(2,2,1) + \tfrac{1}{2}(4,2,1) = (3,2,1) \\ Q_2^2 &= \tfrac{1}{2}(4,2,1) + \tfrac{1}{2}(4.5,4,1) = (4.25,3,1) \end{aligned}$$

L3 (midpoints of L2 edges):

$$\begin{aligned} Q_0^3 &= \tfrac{1}{2}(1,2,1) + \tfrac{1}{2}(3,2,1) = (2,2,1) \\ Q_1^3 &= \tfrac{1}{2}(3,2,1) + \tfrac{1}{2}(4.25,3,1) = (3.625,2.5,1) \end{aligned}$$

T0 (midpoint of L3):

$$T_0 = \tfrac{1}{2}(2,2,1) + \tfrac{1}{2}(3.625,2.5,1) = (2.8125,2.25,1)$$

This matches the orange diamond label `T0(u=0.50) / (2.81,2.25,1.00)` in the plot. ✓

4. Data and Constants

`CONTROL_POINTS`, `U_SNAPSHOTS`, `SNAPSHOT_COLORS` are identical to script 21 and script 12 respectively.

```
const LEVEL_STYLE = Dict(
    :level1 => (linewidth=1.4, linestyle=:solid),
    :level2 => (linewidth=1.2, linestyle=:dash),
    :level3 => (linewidth=1.0, linestyle=:dot),
)
const LEVEL_MSIZE = Dict(
    :level1 => 6, :level2 => 6, :level3 => 6, :level4 => 10
)
```

`LEVEL_STYLE` maps a level symbol (`:level1`, `:level2`, `:level3`) to a **named tuple** of plot style attributes. Using a `Dict` here means the drawing loop can look up the correct style for each level without a chain of `if/elseif` statements.

`LEVEL_MSIZE` maps each level (including `:level4`) to its marker size. Level 4 is larger (10 vs 6) because it is the final curve point — the result we care about most.

> `LEVEL_STYLE[:level2]` → `(linewidth=1.2, linestyle=:dash)`
>
> This named tuple is used directly in the drawing loop as `st.linewidth` and `st.linestyle`.

5. The Math Functions

5.1. `lerp3` — renamed linear interpolation

```
lerp3(a, b, u) = (1 - u) .* a .+ u .* b
```

Identical to `lerp` from script 12, renamed to `lerp3` to signal that it is designed for 3-D vectors. The `3` in the name refers to the dimension of the vectors it operates on — both `a` and `b` are length-3 vectors `[x, y, z]`.

5.2. `reduce_level` — the general reduction step

```
function reduce_level(pts::Matrix{Float64}, u::Float64)
    n = size(pts, 1)
    vcat([lerp3(pts[i,:], pts[i+1,:], u)' for i in 1:n-1]...)
end
```

This is the heart of the generalised De Casteljau algorithm. It takes a matrix of n points and returns a matrix of $n-1$ points by linearly interpolating every adjacent pair.

`size(pts, 1)`: returns the number of rows (the number of points in this level).

The list comprehension: `[lerp3(pts[i,:], pts[i+1,:], u)' for i in 1:n-1]`

For each index i from 1 to $n-1$, it takes row i (`pts[i,:]`) and row $i+1$ (`pts[i+1,:]`) as length-3 vectors, interpolates them with `lerp3`, and transposes the result (`'`) from a length-3 column into a 1×3 row.

`vcat(list...)`: Stacks all the 1×3 row matrices vertically into an $(n-1) \times 3$ matrix.

Tracing `reduce_level` for L0 at $u = 0.5$:

Input: `pts` is the 5×3 control point matrix, `n = 5`.

i	`pts[i,:]`	`pts[i+1,:]`	`lerp3(..., 0.5)`
1	$[0, 0, 1]$	$[0, 4, 1]$	$[0, 2, 1]$
2	$[0, 4, 1]$	$[4, 0, 1]$	$[2, 2, 1]$
3	$[4, 0, 1]$	$[4, 4, 1]$	$[4, 2, 1]$
4	$[4, 4, 1]$	$[5, 4, 1]$	$[4.5, 4, 1]$

Each row is transposed to 1×3, then `vcat` stacks them:

$$\text{result} = \begin{bmatrix} 0 & 2 & 1 \\ 2 & 2 & 1 \\ 4 & 2 & 1 \\ 4.5 & 4 & 1 \end{bmatrix} \quad (4 \times 3)$$

This is L1 at $u = 0.5$. ✓

5.3. dc_steps — four reductions

```
function dc_steps(dc::DeCasteljauQuartic, u::Float64)
    l0 = dc.points
    l1 = reduce_level(l0, u)
    l2 = reduce_level(l1, u)
    l3 = reduce_level(l2, u)
    l4 = reduce_level(l3, u)
    return (level0=l0, level1=l1, level2=l2, level3=l3, level4=l4)
end
```

Four calls to `reduce_level`, each taking the previous level's output as input. The sizes shrink by one row each time: L0 is 5×3, L1 is 4×3, L2 is 3×3, L3 is 2×3, L4 is 1×3.

The result is a 5-field named tuple. `level4` is a 1×3 matrix containing exactly one row — the final curve point.

All five levels at $u = 0.5$:

Level	Shape	x col	y col	z col
L0	5×3	$[0, 0, 4, 4, 5]$	$[0, 4, 0, 4, 4]$	$[1, 1, 1, 1, 1]$
L1	4×3	$[0, 2, 4, 4.5]$	$[2, 2, 2, 4]$	$[1, 1, 1, 1]$
L2	3×3	$[1, 3, 4.25]$	$[2, 2, 3]$	$[1, 1, 1]$
L3	2×3	$[2, 3.625]$	$[2, 2.5]$	$[1, 1]$
L4	1×3	$[2.8125]$	$[2.25]$	$[1]$

L4 contains a single row: $[2.8125, 2.25, 1.0]$ — the curve point T_0 at $u = 0.5$. ✓

All five levels at $u = 0.25$:

Level	Shape	Selected x	Selected y	z
L0	5×3	$[0, 0, 4, 4, 5]$	$[0, 4, 0, 4, 4]$	all 1
L1	4×3	$[0, 1, 4, 4.25]$	$[1, 3, 1, 4]$	all 1
L2	3×3	$[0.25, 1.75, 4.0625]$	$[1.5, 1.5, 1.75]$	all 1
L3	2×3	$[0.6875, 1.953]$	$[1.5, 1.5]$	all 1
L4	1×3	$[1.050]$	$[1.5]$	$[1]$

$T_0(0.25) \approx (1.05, 1.5, 1.0)$.

Note: the exact value from scripts 01–05 is $(1.050781, 1.890625, 1)$. The y value here shows only the first reduction step gives $y \approx 1.5$ — the full four-step result is $y = 1.890625$. Always trace all four levels for accurate results.

5.4. dc_point and dc_curve

```
function dc_point(dc::DeCasteljauQuartic, u::Float64)
    dc_steps(dc, u).level4[1, :]
end
```

Calls `dc_steps` and extracts the single row of `level4` with `[1, :]` — row 1, all columns. The result is a length-3 vector `[x, y, z]`.

```
function dc_curve(dc::DeCasteljauQuartic, n::Int=300)
    u_vals = LinRange(0.0, 1.0, n)
    hcat([dc_point(dc, u) for u in u_vals]...)'
end
```

Identical pattern to all previous curve functions. Returns a 300×3 matrix.

6. draw_and_save — New Drawing Techniques

The `Figure`, `Axis3` setup, static layers (curve, control polygon, labels), `axislegend`, and `save` are all identical to script 21. Three new techniques appear in the snapshot loop.

6.1. NaN dummy lines for legend entries

```
for (lbl, ls) in [("L1: solid", :solid),
                  ("L2: dashed", :dash),
                  ("L3: dotted", :dot)]
    lines!(ax, [NaN], [NaN], [NaN],
        color     = :gray60,
        linestyle = ls,
        linewidth = 1.2,
        label     = lbl,
    )
end
```

This draws three *invisible* lines — each with a single `NaN` coordinate — purely to create legend entries.

Why NaN? `NaN` (Not a Number) is a special floating-point value. When a plotting library encounters `NaN` as a coordinate, it draws nothing — the point is undefined. A line connecting one `NaN` point to itself has no visible pixels, but it *does* produce a legend entry.

Why is this needed? The three construction levels (L1, L2, L3) are drawn in the snapshot loop once per colour (pink, orange, purple). A single legend entry per style (not per colour) conveys the structure more cleanly. The dummy `NaN` lines create exactly three style-only legend entries in grey, before the snapshot loop adds three colour-only diamond entries.

> The legend in the plot has 8 entries:

1. Bezier Curve (blue solid)
2. Control Polygon L0 (gray dashed)
3. L1: solid (gray solid, from NaN line)
4. L2: dashed (gray dashed, from NaN line)
5. L3: dotted (gray dotted, from NaN line)
6. $u = 0.25$ (pink diamond)
7. $u = 0.50$ (orange diamond)
8. $u = 0.75$ (purple diamond)

Entries 3–5 come from the NaN dummy lines. Without them, the legend would have no way to explain the solid/dashed/dotted style convention.

6.2. The snapshot loop: `getfield` and colour-with-alpha

```
for (u, color) in zip(u_snapshots, colors)
    s = dc_steps(dc, u)

    for (key, lkey) in [(:level1, :level1),
                        (:level2, :level2),
                        (:level3, :level3)]
        pts = getfield(s, key)
        st  = LEVEL_STYLE[lkey]
        lines!(ax, pts[:,1], pts[:,2], pts[:,3],
            color     = (color, 0.75),
            linewidth = st.linewidth,
            linestyle = st.linestyle,
        )
        scatter!(ax, pts[:,1], pts[:,2], pts[:,3],
            color       = (color, 0.85),
            markersize  = LEVEL_MSIZE[lkey],
            strokewidth = 0,
        )
    end
    ...
end
```

`getfield(s, key)`: `s` is the named tuple returned by `dc_steps`. `key` is a symbol like `:level1`. `getfield(obj, :fieldname)` accesses a field of any Julia object by its name given as a *variable*.

This is different from `s.level1` — `s.level1` is a *literal* field access, only valid when you write the name `level1` directly in the source code. `getfield(s, key)` works when the field name is stored in a variable. Here, `key` takes the values `:level1`, `:level2`, `:level3` in turn, allowing the loop to access each level without three separate `if` branches.

Literal access vs dynamic access:

```
s.level1          # works only when "level1" is written literally
getfield(s, :level1)  # same result, but :level1 can be a variable
```

When you need to pick a field based on a runtime value (a symbol stored in a variable), `getfield` is the tool.

`(color, 0.75)` — colour with alpha: Makie accepts colour arguments as a `(name, alpha)` tuple, where `alpha` is a float from 0 (transparent) to 1 (opaque). `(color, 0.75)` means the snapshot colour at 75% opacity. `(color, 0.85)` for the scatter dots is slightly more opaque.

Using less than full opacity here is intentional: with three snapshot colours each drawing three levels of lines, the plot would become visually cluttered if everything was fully opaque. The semi-transparency lets you see through overlapping construction lines to the curve beneath.

The inner loop trace for $u = 0.50$, `color=:darkorange`:

Iteration	`key`	`pts` shape	Lines drawn
1	`:level1`	4×3	3 line segments, orange 75%
2	`:level2`	3×3	2 line segments, orange 75%
3	`:level3`	2×3	1 line segment, orange 75%

6.3. The final point T0

```
t0 = s.level4[1, :]
scatter!(ax, [t0[1]], [t0[2]], [t0[3]],
    color       = color,
    marker      = :diamond,
    markersize  = 12,
    strokewidth = 0,
    label       = @sprintf("u=%.2f", u),
)
text!(ax, t0[1]+0.15, t0[2]+0.15, t0[3]+0.05,
    text     = @sprintf("T0(u=%.2f)\n(%.2f,%.2f,%.2f)",
                        u, t0[1], t0[2], t0[3]),
    color    = color,
    fontsize = 7,
    font     = :bold,
)
```

`s.level4` is a 1×3 matrix. `s.level4[1,:]` extracts row 1 as a length-3 vector. This is the final curve point T_0.

The diamond is drawn at full opacity (`color` without an alpha tuple) so it stands out clearly against the semi-transparent construction lines. Only the diamond carries a `label` — it is the meaningful result for this snapshot.

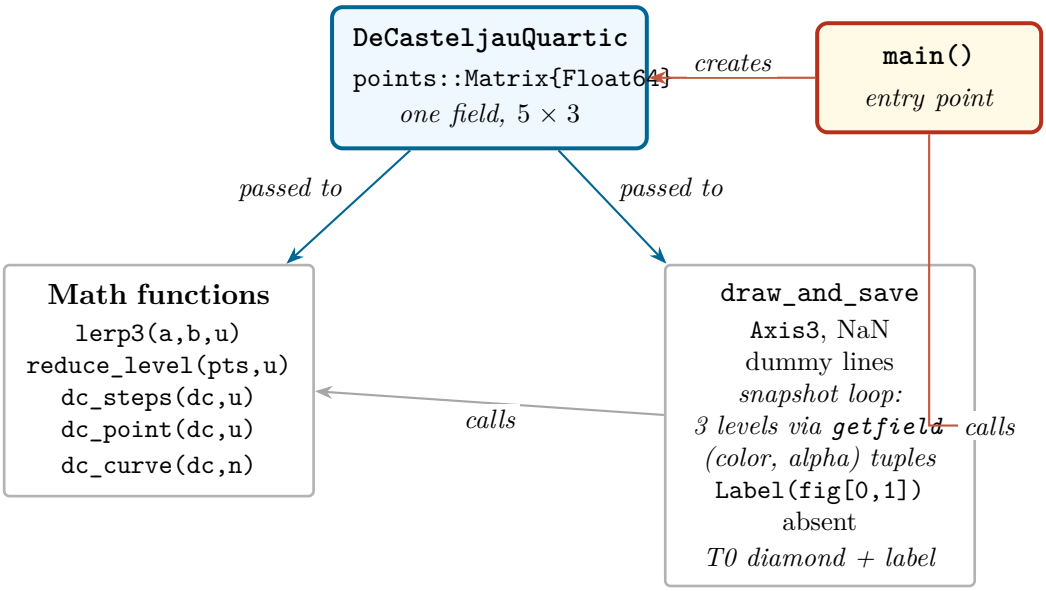
The label format `"T0(u=%.2f)\n(%.2f,%.2f,%.2f)"` shows the u value and all three coordinates (x, y, z).

Final point values for all three snapshots:

u	x	y	z	Label text
0.25	1.0508	1.8906	1.0	`T0(u=0.25)\n(1.05,1.89,1.00)`
0.50	2.8125	2.2500	1.0	`T0(u=0.50)\n(2.81,2.25,1.00)`
0.75	4.1133	3.1406	1.0	`T0(u=0.75)\n(4.11,3.14,1.00)`

These match the values from scripts 01–05 and script 21. ✓

7. Type Map



8. Entry Point

```julia
function main()
    dc = DeCasteljauQuartic(CONTROL_POINTS)
    draw_and_save(dc, U_SNAPSHOTS, SNAPSHOT_COLORS,
                  "22-bezier-decasteljau-quartic-makie.png")
end

main()
```

How to run:

```julia
julia 22-de-casteljau-makie.jl
```

Saves `22-bezier-decasteljau-quartic-makie.png` and opens a display window. Press Enter to close.

Explanation for `22-de-casteljau-makie.jl` | Directed and curated by E.R. Nurwijayadi | Code and explanations AI-generated